CADENCE

A language for writing music, and the compiler that turns it into sound

---

The plain-English guide. No computer science background needed. If you can read a recipe and you have ever heard a piece of music, you can read this.

This document explains what the project is, what every part of it does, and why any of it is difficult — in ordinary words, with an everyday comparison for each idea. There is a glossary at the end, and nothing in the main text depends on knowing a single technical term in advance.

In one paragraph

A compiler is a translator. It takes something a human wrote and turns it into something a machine can act on. Cadence is a compiler whose input is music written as text — "play a D, then an F, then this chord, twice, and put the bass part underneath" — and whose output is a music file that any computer, phone or keyboard can play.

The interesting part is not that it makes sound. It is that turning text into sound correctly requires solving the same seven problems that every compiler on earth has to solve, and this project solves all seven by hand, from scratch, without borrowing anyone else's tools.

What is in this guide

Read it in order if you want the whole story. Jump straight to section 3 if you only want to know what the parts do.

Section	What it covers
1. The big idea	What a compiler is, what this one translates, and why music turned out to be a good choice.
2. The language, line by line	Everything you can write in Cadence, in one table — plus the one feature that shapes the whole design.
3. The seven jobs	The compiler itself, one stage at a time: reading the letters, finding the structure, understanding the meaning, writing simple instructions, making it efficient, safety-checking, and producing the file.
4. The idea I am proudest of	How the compiler proves that making the music faster did not change the music.
5. What the numbers mean	Every measurement the project reports, translated into ordinary words.
6. Trying it yourself	Five commands, and five things worth experimenting with.
7. Glossary	Every technical term used anywhere in this document, in plain English.

> **How to read the seven-jobs section**
>
> Each of the seven stages is laid out the same way: what it does, a comparison from ordinary life, what it actually produces, and what makes it difficult. The comparisons are doing real work — they are not decoration, and each one maps onto the machinery quite exactly.
>
> Nothing in the main text assumes you know any of the terms. Where one is unavoidable it is explained on the spot, and it is in the glossary as well.

1. The big idea

What a compiler actually is

Imagine you have written a recipe in English and you want a factory to produce the dish, ten thousand times, identically. The factory does not read English. It reads a list of machine settings: which valve opens, for how long, at what temperature.

A compiler is the thing in between. It reads the human version, works out exactly what was meant, checks that the instructions make sense, quietly improves them where it can, and writes out the machine version.

Every program you have ever used went through one. When someone writes an app, they write text; a compiler turns that text into the instructions the processor obeys. The text is for people. The output is for machines. The compiler is the translator, and it has to be perfectly faithful — a translator that changes the meaning is worse than useless.

What Cadence translates

Cadence's human side is a small language for describing music. Here is a complete program:

```
tempo 120 ;
key D minor ;
instrument piano = "grand piano" ;

voice tune on piano = [ D4 F4 A4 <D4 F4 A4>/4 ] ;

play tune ;
```

You can probably already guess most of it. Set the speed. Work in the key of D minor. Use a grand piano. Play a D, an F, an A, then all three together for a longer beat. That is the whole point of designing a language: it should read like the thing it is about.

The machine side is a **MIDI file** — the format every keyboard, phone and music program understands. It is not a recording of sound. It is a list of instructions with timestamps: *at moment 0, press key 62 fairly hard; at moment 240, let go.* A player reads the list and produces the notes, the way a pianola reads a punched paper roll.

So Cadence turns readable text into a timed list of key presses. Simple to say. Seven distinct jobs to do.

> **Why a music language, and not a calculator?**
>
> Student compiler projects usually translate arithmetic, because arithmetic is easy to check. The trouble is that arithmetic does not need most of what a compiler course teaches — so those projects end up demonstrating each idea on a toy example, separately.
>
> Music needs all of it at once, for real reasons. Two parts playing at the same time forces the compiler to hand out a limited resource (which turns out to be exactly the hardest problem in real compilers). A repeated section forces it to build a loop. A note written as "the fifth note of the current key" cannot be worked out at all until information from elsewhere in the file reaches it. None of that is contrived. It falls out of what music is.
>
> And there is a bonus: when a compiler for arithmetic has a bug, you get a wrong number. When this one has a bug, you hear it.

2. The language, line by line

Everything you can write in Cadence, in one table. Nothing else exists — that is the whole language.

You write	It means
`tempo 120 ;`	Speed: 120 beats per minute.
`key D minor ;`	Work in D minor. This matters more than it looks — see the next page.
`duration 8 ;`	Any note written without a length is an eighth note.
`velocity 96 ;`	How hard notes are struck, from 0 to 127.
`instrument piano = "grand piano" ;`	Give a nickname to a sound. There are 128 standard ones.
`C4  D#4  Bb3`	Notes by name. The number is the octave; C4 is middle C.
`F4/16  G4/4.`	A note with a length. The dot makes it half again as long.
`^1  ^5  ^3'  ^2,`	"The first note of the key", "the fifth", and so on. The marks move it up or down an octave.
`r/8`	A rest — silence for an eighth of a beat.
`<C4 E4 G4>/4`	A chord: three notes struck together.
`[ ... ]`	Brackets, to group things.
`repeat 4 of X`	Do X four times.
`transpose -12 of X`	Play X twelve semitones lower.
`slow 2 of X`	Stretch X to twice the length. There is a fast too.
`phrase riff(t) = ... ;`	Name a passage so you can reuse it, optionally with a setting you pass in.
`voice lead on piano = ... ;`	A part, played on one instrument.
`play lead | low ;`	Play these parts. The bar means at the same time.

The one feature worth understanding properly

Look again at ^5. It does not say which note to play. It says *the fifth note of whatever key we are in*. In D minor the fifth note is an A. In C major it is a G. So the compiler cannot turn ^5 into a key press until the key — written at the top of the file, possibly hundreds of lines away — has travelled down to it.

This sounds like a small notational convenience. It is the reason one whole stage of the compiler has to exist, and it has a nice consequence:

```
key A minor ;                       // change this one word ...
phrase up = [ ^1 ^2 ^3 ^4 ^5 ] ;    // ... and every note here moves
```

Change minor to major and the melody changes, without editing a single note. The information flows downhill through the structure of the file, and the notes are computed from it. Programmers call the mechanism inherited attributes; a musician would call it "reading in context", which is the same thing.

3. The seven jobs, one at a time

What follows is the whole compiler. Each stage gets: what it does, a comparison from ordinary life, what it actually produces, and what makes it hard. They run in order, each one feeding the next.

#	Stage	In one line
1	Reading the letters	Chop the text into words.
2	Checking the sentences	Work out the structure — what belongs inside what.
3	Understanding the meaning	Resolve names, fill in context, catch mistakes.
4	Writing simple instructions	Rewrite everything as tiny uniform steps.
5	Making it efficient	Remove waste, without changing the result.
6	Safety-checking	Prove the instructions cannot misbehave before running them.
7	Producing the file	Hand out instruments, then write the bytes.

Job 1 — Reading the letters

What it does

The compiler receives one long line of characters: `t, e, m, p, o, space, 1, 2, 0, space, semicolon`. Before anything else can happen, that has to become a list of **words**: the word tempo, the number 120, the punctuation mark.

The everyday version

You do this constantly and never notice. Reading the string "theredressisonthebed" is hard; reading "the red dress is on the bed" is effortless. Splitting a stream of letters into words is a real task, and it has real ambiguities — that same string could be "the red dress" or "there dress".

What makes it hard here

In Cadence the single letter `r` means a rest. But `riff` is a name someone chose for a passage. If the compiler reads `riff` one letter at a time it would see `r` — a rest! — followed by the mysterious `iff`. So it uses one firm rule: **always take the longest word that could possibly fit**. Four letters beat one, so riff wins. When two possibilities tie on length, whichever rule was written first wins.

Those two rules — longest match, then declaration order — are exactly how the professional tool for this job behaves, and they are the entire tie-breaking policy.

How it is built

You describe each kind of word as a pattern ("a capital letter A to G, optionally a sharp or flat sign, then a digit"). The compiler converts all of those patterns into a single machine with 47 states — a flowchart it walks through, one letter at a time, that always knows whether it has just finished a valid word. Building that machine from the patterns is a classic algorithm, and this project implements it rather than calling a library for it.

> **A small point of pride**
>
> Nearly every program that needs to match patterns uses a ready-made pattern engine. Python ships with

> one. This project deliberately does not use it — the pattern-to-flowchart construction is one of the things being demonstrated, so borrowing it would be like handing in someone else's homework.
>
> It was checked against the textbook example every course uses, and produced exactly the four-state machine printed in the notes. That check runs automatically every time, so it cannot quietly break later.

Job 2 — Checking the sentences

What it does

Now the compiler has words. It needs the structure: which things are inside which. In the phrase "repeat 2 of [D4 F4]", the repeat owns the brackets, and the brackets own the two notes. That nesting is what determines what actually happens.

The everyday version

Sentence diagrams from school. "The dog that chased the cat barked" — you know that "that chased the cat" belongs to the dog, not the barking, and you know it from grammar, not from meaning. Same job here.

What makes it hard

Grammar can be **ambiguous**: genuinely readable two different ways. This happened in this project, and it is worth telling honestly.

Originally, chords and groups both used ordinary round brackets. So when the compiler had read the characters "(D4" it was stuck: it could not tell whether it was inside a chord (all notes together) or a group (notes one after another). It would only find out at the closing bracket — too late, because by then it has already had to make the decision.

The compiler-building machinery detects this automatically and reported 26 separate places where the grammar was undecidable. The fix was not to paper over it. **The fix was to change the language**: chords now use angle brackets, `<C4 E4 G4>/4`, so the very first character settles the question. The broken version of the grammar is kept in the project so the report can show the conflicts appearing and then disappearing.

Two parsers, because one is not convincing

A structure-finder that does not crash is not the same as one that is correct. So this project contains two completely different ones, written using two different techniques from the same grammar, and after every compilation it dumps the structure found by each and compares them character for character. They agree on every example. If they ever disagree, the build says so immediately and one of them has a bug.

That is a professional testing technique (engineers call it differential testing), and it costs nothing but the effort of building the second one.

Job 3 — Understanding the meaning

What it does

Structure is not meaning. This stage answers questions like: does the instrument named here actually exist? Does this passage take one setting or two? What actual note is "the fifth of the key"? How long is this note, given that its length was never written down?

The everyday version

Proofreading, as opposed to spell-checking. "Colourless green ideas sleep furiously" passes every grammar rule and means nothing. "Please deliver it to the address above" is perfectly grammatical and useless if there is no address above.

Information flowing two ways

This is where the earlier point about context becomes machinery. Some facts travel downwards, from the outside in:

- the key and mode, so that "the fifth note" can be worked out
- the current octave, so that a bare D knows which D
- the default length, for notes written without one
- how hard to strike, and any transposition or stretching applied by an enclosing instruction

And some facts travel upwards, from the inside out:

- how long this passage lasts in total
- how many notes are inside it
- the highest and lowest note it reaches, which is how the compiler can warn that a transposition has pushed something off the end of the keyboard

Both directions in a single pass over the structure. Getting information to flow down as well as up is the part that requires care, and it is the specific thing this stage of a compiler course is about.

What it catches

The mistake	What the compiler says
A name that was never declared	'plano' is not a declared instrument — add: instrument plano = 0 ;
Wrong number of settings passed in	phrase 'riff' takes 1 argument(s) but got 2 — it is declared on line 16 as riff(t)
A passage that plays itself	phrase 'loop' is recursive (loop -> loop) — a phrase cannot play itself: the music would never end
An instrument name with a typo	unknown instrument name 'grand pianoo' — did you mean: 'grand piano', 'piano'?
A speed no instrument could play	tempo 900 is outside the playable range 20-400
A transposition that leaves the keyboard	pitch 198 is outside MIDI range 0-127 — check the transpose amounts

Every message carries the line, the column, the offending line of your file, and a small arrow pointing at the exact character. The suggestion in the fourth row is computed by measuring how many letters would have to change to turn what you typed into each of the 128 real instrument names, and offering the closest. That is the same trick the compilers professionals use every day play on them.

> **A bug this stage taught me**
>
> A passage that plays itself would make the compiler follow it into itself, for ever — and the first version did exactly that, crashing instead of complaining.
>
> The fix was ordering. The compiler now maps out which passages call which, finds the loops in that map, and reports them BEFORE it starts examining the passages in detail. Then it refuses to descend into anything

> already known to be circular. A test now guards it, so the crash cannot come back.

Job 4 — Writing simple instructions

What it does

The compiler now understands the music, but it understands it as a nested structure, and nested structures are awkward to optimise and awkward to execute. So it rewrites everything as a long flat list of tiny steps, each doing exactly one thing.

The everyday version

The difference between "make a cup of tea for everyone" and a numbered list: fill kettle; switch on kettle; wait; put bag in cup; pour; wait three minutes; remove bag. Boring, repetitive, much longer — and impossible to misunderstand. That is the trade you want before handing work to a machine.

What one note becomes

six steps for a single note

```
t2 = 62 + $off        // the note, plus any transposition
NOTEON  t2, vel 96    // press the key
t3 = 240 * $sn        // how long, times the stretch we were given ...
t4 = t3 / $sd         // ... divided by the other half of the stretch
DELAY   t4            // hold it
NOTEOFF t2            // let go
```

The deliberately silly part

Those six steps are emitted for *every* note — including notes with no transposition and no stretching, where the compiler is carefully adding zero and multiplying by one. That looks like incompetence. It is standard practice, and it is on purpose.

The rule in compiler design is: the front half generates code uniformly and stupidly; the back half cleans it up. Trying to be clever in both places at once produces code nobody can maintain and bugs nobody can find. It also has a happy side effect for this project — the next stage has genuine work to do, so the improvement figures reported later are real measurements rather than a rigged demonstration.

The clever part

"Repeat this four times" becomes a loop, and a loop needs a jump: *if we are done, jump past the end*. But at the moment that jump has to be written, the compiler does not yet know where the end is — it has not generated it. So it writes the jump with the destination left blank, notes down where it left the blank, and fills it in later when the destination finally exists. This is called backpatching, and it is the same problem as writing "see page ___" in a book you are still writing.

Job 5 — Making it efficient

What it does

Thirteen separate clean-up passes run over that flat list, each looking for one specific kind of waste.

The pass	What it looks for	In plain words
common subexpressions	the same sum computed twice	You already worked that out. Reuse the answer.

The pass	What it looks for	In plain words
inlining	a passage called from one place	Instead of "go and do the riff", paste the riff in.
constant folding	sums of known numbers	2 + 3 becomes 5 now, not while playing.
copy propagation	values shuffled between slots	Stop moving it around and use the original.
loop unrolling	a loop that runs a known number of times	"Do this twice" becomes the thing, written twice.
strength reduction	expensive arithmetic	Doubling by adding rather than multiplying.
dead code elimination	work whose result is never used	If nobody reads it, do not compute it.
peephole	silly adjacent pairs	"Wait 0 seconds", or a jump to the very next line.

They run in a deliberate order, and some run more than once, because each one creates opportunities for the others: inlining makes settings into fixed numbers, folding those numbers turns instructions into simple copies, removing the copies makes other instructions pointless, and removing those is the biggest win of all.

The number that surprised me

Optimisation is supposed to make things smaller. Here are the actual figures for the showcase example:

	Before	After	Change
Instructions that exist in the file	173	247	43% BIGGER
Instructions actually carried out when playing	631	241	62% FEWER

The code got bigger and the work got smaller, and both of those are the same decision. Pasting a passage in four times instead of jumping to it four times costs space and saves all the jumping and bookkeeping. Compilers make this trade constantly.

The lesson: if I had reported only the size, my optimiser would look like it had failed. If I had reported only the work, it would look free. So the project measures both, after every single pass, by *actually running* the music and counting.

> **How measuring caught a silent bug**
>
> One pass hunts for sums that have already been computed. It reported finding none — on a passage that obviously computes the same multiplication three times over.
>
> The cause was a one-line ordering mistake: the pass recorded each new result in its notebook, and then immediately crossed out everything associated with the slot it had just written to — including the note it had only just made. So its memory was wiped every single time and it could never find a repeat.
>
> Swapping two lines fixed it and it instantly found fifteen. The important part: a pass that finds nothing looks exactly like a pass that is broken. Without a table printing what each pass achieved, this would have shipped, silently doing nothing at all.

Job 6 — Safety-checking, before anything runs

What it does

The instruction list is now machine-ready. Before executing a single step, the compiler examines it and proves five things about it — without running it.

It proves	In plain words
Every jump lands somewhere real	No instruction can send us off the edge of the list.
Every call gets its arguments	Nothing is invoked with fewer settings than it needs, on any possible route through the code.
Parallel sections are properly nested	Every "play these together" has a matching close, and the two sides of a fork agree about how deep they are.
Nothing is read before it is written	No step uses a value that might never have been given one, on any route.
Every passage ends by returning	Control cannot run off the bottom into whatever happens to be next.

The everyday version

An aircraft checklist. You do not discover the fuel line is disconnected by taking off and seeing what happens; you check on the ground. "On any possible route" is the phrase that matters — the check has to hold for every path the music could take, including paths that this particular piece happens never to use.

Why this is a real thing, not an exercise

This is exactly what happens when you open a web page that runs a program, or when your phone loads an app: the code arrives from someone you do not trust, and something inspects it and proves it cannot misbehave before letting it run. The Java virtual machine does it. Web browsers do it. The Linux kernel does it. Same family of checks, much smaller scale here.

The checking is done twice — once on the plain instructions and once after all thirteen optimisation passes have rewritten them. The reason is unflattering and correct: my optimiser is also just a program I wrote, and it could be wrong.

Job 7 — Handing out instruments, and writing the file

The scarce resource

MIDI has sixteen channels. Think of them as sixteen musicians. Each can play one instrument at a time. Two parts playing simultaneously need two musicians; two parts that take turns can share one.

So: work out when each part is actually sounding, draw a diagram joining every pair that overlaps, and then colour that diagram so that no two joined parts get the same colour. Colours are channels.

Why this is the interesting one

This is the hardest problem in real compilers, wearing a costume. A processor has a small number of extremely fast storage slots called registers, and the compiler must decide which of a program's thousands of values live in them. Two values needed at the same moment cannot share a slot. The industry solution is precisely this: build the overlap diagram and colour it. It is the **same algorithm**, and this project uses the standard one.

Two details make it more than an analogy:

- **Percussion is forced.** In the MIDI standard, drums must be on channel 10. So that part is coloured before the algorithm starts — exactly what a real compiler does with a value that has to live in one specific register.

- **Running out is possible.** One example asks for seventeen parts at once. Fifteen channels are usable. The algorithm cannot colour it, so it spills: it picks the two parts with the fewest notes, makes them share, and warns you that they may cut each other off. In a normal compiler spilling makes your program slower. Here, you can hear it.

Writing the bytes

Finally the file itself, assembled by hand, one byte at a time — no music library involved. The only genuinely fiddly bit is that gaps between notes are stored in a compressed form where each byte carries seven bits of the number and one bit meaning "there is more of this number coming". A gap of 240 is stored as two bytes rather than four.

To be sure the output is real, the test suite contains a MIDI reader written separately from the writer. It walks through every file the compiler produces, checks the structure, and confirms that every note that starts also stops, and that the counts match what the compiler claimed. Two people who have never met, agreeing about a document, is stronger evidence than one person checking their own work.

4. The idea I am proudest of

Everything above has a hole in it, and it is worth stating plainly.

Thirteen passes rewrite the music's instructions. Each one is supposed to leave the result identical. But *supposed to* is doing a lot of work in that sentence. A subtly wrong pass would not crash; it would just quietly produce slightly different music, and I might not notice for weeks.

So the compiler checks

Before any optimisation, the compiler plays the piece internally and writes down every single event: which note, how hard, at what moment, in which part. Not the sound — the exact list of instructions, which is what really matters.

Then, after each of the thirteen passes, it plays the piece again and compares the new list against the original, event by event.

real output, showcase example

```
pass                    instructions   carried out   music unchanged
common subexpressions   173 -> 173         631            yes
inline phrases          173 -> 276         542            yes
constant folding        276 -> 276         542            yes
copy propagation        276 -> 276         542            yes
loop unrolling          276 -> 567         462            yes
dead code elimination   567 -> 325         241            yes
drop unused passages    325 -> 247         241            yes
```

If any pass ever changes what you would hear, that last column says no and the build fails. This runs for every example, at every optimisation level, on every single run, and it is asserted by the automated tests.

Why this is the right way round

The usual way to gain confidence in an optimiser is to look at the code carefully and convince yourself. That works right up until it doesn't.

This is different in kind: the compiler is not claiming to be correct, it is **demonstrating** it, on the actual input, every time it runs. It is the difference between a chef promising the recipe is right and a chef tasting the dish.

There is an honest complication, and hiding it would be cheating. Two of the passes are supposed to change the sound slightly — they smooth over the tiny gaps between repeated identical notes, which is a musical choice, not an error. Those two are switched off by default and named explicitly as exceptions in the comparison. An invariant with a documented exception is engineering; an invariant with a quiet exception is a lie.

> **If you remember one thing from this document**
>
> This project does not ask to be trusted. Two independently built parsers must agree on the structure. A verifier must approve the instructions before they run, and again after they are optimised. The music must come out identical after every single optimisation pass. An independently written reader must be able to parse the output file. 72 automated tests check all of it on every run.
>
> That habit — build the thing, then build the second thing that catches the first thing lying — is most of what separates working software from software that appears to work.

5. What the numbers mean

The project reports a lot of measurements. Here is what each one is actually telling you, so the report and the slides can be read without a course behind you.

The measurement	Value	What it means in plain words
Word patterns	25	Twenty-five kinds of word exist in the language: notes, numbers, names, punctuation, and so on.
States in the word-recognising machine	47	The flowchart the compiler walks through, one letter at a time, to spot where words end. Built automatically from the 25 patterns.
Character groups	37	Instead of 256 separate columns for every possible character, characters that behave identically are grouped. The ten digits are one column, not ten.
Grammar rules	56	Fifty-six rules describe every legal arrangement of words. The entire language is defined by these.
Structure-finder size, three ways	114 / 204 / 114	The same grammar, prepared by three standard methods. The middle one is the most powerful and needs 204 states; the third gets the same power in 114 by merging states that behave alike. That saving is the whole reason the third method is the industry default — and here it is measured, not quoted.
Undecidable spots	0	Nowhere in the language can the compiler get stuck between two readings. The first draft of the grammar had 26 such places.
Instructions written	173 -> 247	The flat list got 43% longer, on purpose, to make the next number smaller.
Instructions carried out	631 -> 241	Playing the piece now takes 62% less work. This is the number that matters.
Music changed	no	Verified after each of the thirteen clean-up passes.
Parts / channels / spills	4 / 4 / 0	Four parts, four musicians, nobody had to share. The stress-test example asks for seventeen and two have to share.
Automated tests	72	Checks that run in about a second and re-verify everything above.
Outside code used	0	No borrowed parser, no borrowed pattern engine, no borrowed music library.

6. Trying it yourself

There is nothing to install beyond Python itself, and no internet connection is needed at any point.

1. On Windows, double-click `run.bat`. A window opens with the music on the left and seventeen tabs on the right — one for each thing the compiler worked out. Edit the music, press F5, and watch every tab change.
2. Prefer the command line? `python -m cadence examples/07_showcase.cad` writes a .mid file next to it. Double-click that to hear it.
3. To see everything at once in a browser: add `--report r.html`. That single file contains every table the compiler built, including a picture of the finished music. It needs no internet and no plug-ins.
4. To watch the compiler talk to itself: add `--dump all`. Every stage prints.
5. To check I am not making any of this up: `python -m unittest discover -s tests -v`. Seventy-two checks, about a second.

Things worth trying, in order of how interesting the result is

- Open `examples/02_scales.cad` and change `minor` to `major`. Every note changes and you edited none of them.
- Compile the same file twice, once with `-O0` and once with `-O2`. The instruction lists look wildly different. The two .mid files are byte-for-byte identical.
- Compile `examples/06_spill.cad` and read the channel tab. Seventeen parts, fifteen musicians, and you can see which two were made to share.
- Break something on purpose — delete a semicolon, misspell an instrument — and read the complaint. Then look at the three `bad_*.cad` files, which are broken deliberately, one for each way of being wrong.
- Write eight notes of something you know. It will probably work first time, which is the real test of whether the language design was any good.

7. Glossary

Every technical term this document uses, in one place, in ordinary words.

Term	Plain English
compiler	A translator from what a human wrote to what a machine can carry out.
token	A word, in the compiler's sense: a note, a number, a name, a bracket.
state machine	A flowchart the compiler walks through one letter at a time to find where words end.
grammar	The rules saying which arrangements of words are legal, and what belongs inside what.
parser	The part that applies the grammar and works out the structure.
ambiguous	Readable two different ways. Fatal in a language; the compiler-building tools detect it automatically.
syntax tree	The structure, drawn as a tree: the outermost thing at the top, individual notes at the bottom.
attribute	A fact attached to part of the structure. Some are passed down from outside (the key), some are worked out and passed up (how long this lasts).
semantic analysis	Checking meaning rather than form. Proofreading, not spell-checking.
symbol table	The compiler's address book: every name, what kind of thing it is, where it was declared, whether anyone used it.
intermediate code	The flat list of tiny steps that sits between understanding the music and producing the file.
backpatching	Writing a jump whose destination is not known yet, and filling it in later. "See page ___".
basic block	A run of instructions with no jumps in or out — once it starts, it all happens.
optimisation	Removing waste without changing the result.
data-flow analysis	Working out what is true at each point in the code by going round and round until nothing changes any more.
dead code	Work whose result nobody ever reads. Safe to delete.
inlining	Pasting a short passage in where it is used instead of jumping away to it.
unrolling	Writing out the body of a loop several times instead of looping.
virtual machine	A pretend processor, written in software, that carries out the instruction list. Handy because you can watch it.
verifier	The safety inspector that proves the instruction list cannot misbehave, without running it.
activation record	The scrap of memory a passage gets while it is playing: where to return to, and its own settings and workings.
register allocation	Deciding which values live in the processor's few fast slots. Here: which part gets which MIDI channel.

Term	Plain English
spilling	What happens when there are not enough slots. Somebody has to share.
MIDI	A file of timed instructions — press this key now, release it then — rather than recorded sound. Like a pianola roll.
velocity	How hard a key is struck, 0 to 127. Nothing to do with speed.
tick	The compiler's unit of time. 480 of them make one beat.
differential testing	Building the same thing twice, two different ways, and checking they agree.
